CLAUDE.md — working memory for this project

Read this first. It's a fast-orientation file for an AI agent (or Will) picking this project back up, not a replacement for the real docs.

What this project is

A wagering-strategy model and practice tool for Jeopardy!'s Daily Double and Final Jeopardy decisions. 3-player primary case; 2 and 1 players are reduced cases of the same model. Full explanation: `docs/white-paper.md`.

Two-tier design (memorize this, it comes up everywhere)

- **Tier 1** (`engine/fj_wager.py`) — worst-case guarantee, no confidence input needed. "What wager can't hurt me if I'm right?"
- **Tier 2** (`engine/fj_bestresponse.py` for FJ; `engine/simulate.py` + `engine/dd_wager.py` for Daily Double) — confidence-weighted, plays against simulated contestants (`engine/behavior.py`, ADR-0003): role-based wager mixes, 50% opponent accuracy, outcomes correlated at 0.3 across all players. Some values are PUBLISHED (Tesauro et al., 2013), the rest are marked ASSUMED. "Given my real confidence, what maximizes my win odds?"

Tie rule: since 24 Nov 2014 a tie for first goes to a tiebreaker clue. Tier 2 and the DD engine score a k-way tie as 1/k of a win; nobody wins at $0. Tier 1 requires an outright beat. Tier 2 can recommend less than Tier 1 at low confidence (a big bet loses more when you miss) — not a bug.

Known gotchas

- **Git and GitHub work.** `origin` is https://github.com/willhoop/jeopardy-wagering. The live app is at https://willhoop.github.io/jeopardy-wagering/app/. CI (`.github/workflows/ci.yml`) runs pytest and `tests/js_parity.js` on every push and pull request.
- **`app/index.html` is a hand-written JS port of the Python engine.** If you change the Python engine, update the JS too. `node tests/js_parity.js` checks the exact functions against the hand-derived values; Monte Carlo output is checked for invariants only. The app's `BEHAVIOR` object mirrors the configuration block in `engine/behavior.py` — change both. Risk #8 in `RISK_REGISTER.md`.
- **Never use the jwolle1 historical dataset (or anything scraped from J-Archive) in a shipped artifact.** Internal validation only, if at all. See ADR-0001.
- **Every doc gets a PDF companion in `build/`.** pandoc is not installed. Use `python build/md_to_pdf.py <in.md> <out.pdf>` (Python `markdown` + WeasyPrint; falls back to headless Edge/Chrome when WeasyPrint's GTK libraries are missing, as on this Windows machine). Regenerate the PDF in the same pass as any Markdown edit.

Commands

```
# Run tests
cd tests && python -m pytest -v
# JS/Python parity
node tests/js_parity.js

# Rebuild a doc's PDF
python build/md_to_pdf.py docs/white-paper.md build/white-paper.pdf
```

Where things stand (see CHANGELOG.md for the authoritative, dated version)

Run `git log --oneline -5` and read the top `CHANGELOG.md` entry for the current version and state. As of 2.0.0 (2026-10-13): Tier 1 + Tier 2 FJ engines, the Daily Double Monte Carlo engine, simulated contestant behavior with the tiebreaker rule, the interactive app (live), the white paper, the deck, the Diátaxis docs, ADRs 0001-0003, the risk register, PDF companions, CI, and `llms.txt`.